

项目地址

[FPSAssignment](#)

游戏整体流程(Game Loop)

在限定时间内击杀Enemy AI，击杀AI会获得分数

GAS输入绑定

本质上就是给由Input来trigger触发的Ability绑定事件回调函数，其实现的方式如下

1. 与其他Ability一样，首先会将其存入Data Asset中，其会与一个Input Tag对应
2. 在加载Data Asset时，会通过Ability System Component将Ability给予角色，在给予角色前，会将InputTag加入到AbilitySpec的DynamicSpecSourceTags容器中，使得InputTag可以和Ability建立对应关系
3. 在上述基础之上，事件回调函数就可以统一，本质上就是按下(pressed)就激活该Ability，然后松开(released)就移除该Ability，如何找到对应的Ability？上述的InputTag就发挥了作用，只需要找到与当前输入的InputTag对应的Ability即可（现在的实现方式存在着性能问题，通过 `GetActivatableAbilities` 来获取所有可激活的Ability，然后找到与InputTag匹配的Ability，算法复杂度为 $O(n)$ ，该部分的性能开销会随着Ability数量的增加而增加，有待优化）

武器的管理

对于武器，我们将其分为了敌人的武器（其在敌人生成时就直接装备）和玩家的武器（武器初始状态下不会装备，需要玩家通过输入来触发武器的装备）

敌人和玩家的武器有一些公有的部分（比如StaticMeshComponent），但是对于玩家武器来说，其需要一些附属在武器上的Weapon Data，比如又Weapon带来的Anim Layer（因为装备与否其动画是不一样的）和Weapon带来的额外的Ability（开火的Ability等），因此可以把公有的部分抽象成一个基类，然后派生出一个PlayerWeapon

对于武器的管理，我们通过给角色增加一个Component进行武器的管理，其中通过一个Map数据结构通过GameplayTag来查询对应的武器，然后在武器被生成时就会调用该Component的武器注册函数将其存入Map中，同样的因为需要同时管理敌人Weapon和Player Weapon，但是其武器注册的逻辑是公有的，因此可以将其放在基类Component，然后派生出一个PlayerComponent来实现其特有的Getter来获取PlayerWeapon

动画资产复用

在Unreal的FPS的模板中，仅提供了Equip武器的Animation Sequence，并没有提供Unequip的Animation Sequence，但是我们起始可以通过倒放Equip的动画来获得Unequip的动画

1. 创建原始Animation Sequence的副本
2. 然后设置其Play Rate为-1就可以实现倒放
3. 然后基于此动画资产创建Anim Montage实现后续的Unequip

AnimLayer

角色往往存在多套动画，比如角色在不装备武器时使用一套动画，在装备武器时使用另一套动画，为了提高角色动画系统的可扩展性，引入了AnimLayer的概念，通过此设计，不同的Anim可以动态的装载(Link)和卸载(Unlink)，从而动态的实现不同动画之间切换，具体实现流程如下

1. 创建Animation Layer Interface，在其中定义所有的AnimLayer，但是并不会给出实现这些定义的AnimLayer
2. 然后通过创建一个继承自自定义的AnimLinkedLayer class的Anim Blueprint，我们将其称为Base_AnimLayer，并实现上述Animation Layer Interface，给这些AnimLayer一个具有普适性的实现，将实际的动画资产的填充下放给实际的AnimLayer
3. 而对于真正的AnimLayer则继承自上述Base_AnimLayer，并填入实际的动画资产
4. 在WeaponData中只需维护一个AnimLinkedLayer类的指针，并填入相应的AnimLayer即可，然后在装备武器后，从WeaponData中找到AnimLayer，并将其进行Link即可

5. 此外主AnimBP中，需要实现上述Animation Layer Interface，然后根据实际情况（比如通过是否装备武器的branch）来决定是否要使用AnimLayer来作为当前的动画

参考文档：[UE5 白话Lyra动画系统](#)

属性设计(Attribute)

角色属性

与角色绑定的属性放在AttributeSet中(GAS中用于维护角色属性的数据结构)

- 生命值
- 防御值

武器属性

与武器绑定的属性，可以将其分为静态属性和动态属性

静态属性

- 武器伤害：可以放在WeaponData中，决定该武器能够造成的伤害
- 武器的弹夹容量：可以容纳的弹药上限

动态属性

- 武器的弹药数量：当前弹夹中剩余的弹药数量，会随着武器的开火、装弹等行为动态发生变化，可以作为武器的成员，然后由Combat Component提供接口来动态改变该属性

UI和角色之间的数据通信

对于角色的UI，比如说血条的显示，其需要依赖角色的生命值属性，当角色的生命值发生变化时，就需要通过某种方式将此信息传递给UI，使其能够根据此信息来更新UI

类似于对武器的管理，同样可以通过一个UIComponent来对角色和UI之间的数据通信进行管理，本质上就是在此Component中定义一系列Delegate，然后在UI端会listen这些Delegate并实现相应的回调函数来更新UI，在Character端，当属性发生更新时就会对Delegate进行Broadcast，将属性更新的信息传递给UI，这样就实现了从Character到UI的数据通信

射击状态设计

射击状态可以分为非瞄准状态下的射击(Dry Fire)和瞄准状态下的射击(Aim Fire)

当玩家按下鼠标的右键就会进入瞄准状态，此时玩家会拥有一个Gameplay Status Tag表示玩家处于瞄准状态，只有在拥有该Status Tag下才能够触发瞄准射击的Ability，通过指定Gameplay Ability的Activation Required Tag来实现

此外在瞄准状态下，将无法激活Dry Fire，通过设置Dry Fire Ability的Activate Blocked Tag来实现

弹药管理

枪械的弹药应该与武器绑定在一起，主要用于进行弹药数量的管理，用于处理子弹发射和重新装填子弹的子弹数量的动态管理

由于子弹这个对象是频繁创建和销毁的，如果频繁的调用SpawnActor来生成子弹对象，调用Destory来销毁不需要的子弹对象，就会因为频繁的创建和销毁产生额外的开销，因此可以通过对象池来优化，通过设计一个子弹对象池的组件挂载在武器上，武器的进行发射时会向该子弹索要空闲的子弹对象，然后当子弹要进行销毁操作时，不会将其真正销毁，而是将其状态设置为空闲状态；此外，当子弹数量的需求超过了对象池当前的容量时，需要进行动态扩容，即需要创建新的子弹对象加入到对象池中。

整体流程如下

1. 首先会在PoolComponent中创建对象池的所有的子弹，子弹类的BeginPlay函数中会绑定子弹的碰撞盒的碰撞回调函数，即只需要绑定一次就可以反复使用此回调函数
2. 射击时，通过武器的PoolComponent来获取子弹，通过子弹的SetActive传入必要的参数，比如说Instigator信息，以及子弹生成的位置、Rotation信息
3. 在SetActive函数中根据传入的参数将选中的空闲子弹设置为激活状态，将其发射出去，并设置子弹的生命周期Timer，当Timer计时结束后就自动回收子弹
4. 当子弹命中Actor，就会向Instigator发送事件，供射击者进行击中处理，然后子弹被回收

Sound Concurrency

在播放音效时，为了避免多个相同的音效，同时播放，导致音效显得十分杂乱，可以通过配置Sound Concurrency来限制同时并发播放的音效数量

伤害计算与施加

伤害本质上就是对Attribute进行修改，在GAS框架下，通常通过GameplayEffect来实现

由于涉及到相对复杂的计算，无法直接通过Modifier来直接进行修改属性，需要通过Execution来处理相对复杂的属性计算

通过定义派生自 `UGameplayEffectExecutionCalculation` 的类，实现自己的计算逻辑，在 `Execute_Implementation` 函数中实现具体的计算逻辑

首先需要定义 `AttributeCapture`，用于捕获计算所需的属性信息，在计算伤害时需要Target的防御力属性，以及需要Target的DamageTaken属性用于最终的伤害输出

```
struct FFPSDamageCapture
{
    DECLARE_ATTRIBUTE_CAPTUREDEF(DefensePower)
    DECLARE_ATTRIBUTE_CAPTUREDEF(DamageTaken)

    FFPSDamageCapture()
    {
        DEFINE_ATTRIBUTE_CAPTUREDEF(UFPSAttributeSet,
        DefensePower, Target, false)
        DEFINE_ATTRIBUTE_CAPTUREDEF(UFPSAttributeSet, DamageTaken,
        Target, false)
    }
};
```

然后通过单例模式来获取该struct的唯一实例

```
static const FFPSDamageCapture& GetFPSDamgeCapture() // 返回局部静态变量的引用，Mayer单例模式
{
    static FFPSDamageCapture FFPDamageCapture;
    return FFPDamageCapture;
}
```

在构造函数中将这些属性加入到 `RelevantAttributesToCapture` 中

```
UGExecCalc_DamageTaken::UGExecCalc_DamageTaken()
{

    RelevantAttributesToCapture.Add(GetFPSDamgeCapture().DefensePowerDef);

    RelevantAttributesToCapture.Add(GetFPSDamgeCapture().DamageTakenDef);
}
```

接着就是在 `Execute_Implementation` 中执行计算，在 `ExecutionParams` 中的 `GameplayEffect` 获取 `SetByCallerTagMagnitudes`，这是一个 `TMap<FGameplayTag, float>`，存储着在构造 `GameplayEffect` 时存储的额外信息，在此场景下，存入了武器的伤害值

此外通过 `ExecutionParams` 的 `AttemptCalculateCapturedAttributeMagnitude` 获取捕获的防御值属性，然后进行计算

最后通过 `OutExecutionOutput` 的 `AddOutputModifier` 将最终的结果进行写入

```
void UGExecCalc_DamageTaken::Execute_Implementation(const
FGameplayEffectCustomExecutionParameters& ExecutionParams,
FGameplayEffectCustomExecutionOutput& OutExecutionOutput) const
{
    const FGameplayEffectSpec& EffectSpec =
    ExecutionParams.GetOwningSpec();

    FAggregatorEvaluateParameters EvaluateParameters;
```

```

    EvaluateParameters.SourceTags =
EffectSpec.CapturedTargetTags.GetAggregatedTags();
    EvaluateParameters.TargetTags =
EffectSpec.CapturedTargetTags.GetAggregatedTags();

    float weaponBaseDamage = 0.f; // 武器的基础伤害

    for (const TPair<FGameplayTag, float>& TagMagnitude :
EffectSpec.SetByCallerTagMagnitudes)
    {
        if
(TagMagnitude.Key.MatchesTagExact(FPSGameplayTags::Shared_SetByCal
ler_WeaponBaseDamage_Rifle))
        {
            WeaponBaseDamage = TagMagnitude.Value;
        }
    }
    float TargetDefensePower = 0.f;
    ExecutionParams.AttemptCalculateCapturedAttributeMagnitude(
        GetFPSDamageCapture().DefensePowerDef,
        EvaluateParameters,
        TargetDefensePower
    );

    checkf(TargetDefensePower > 0.f, TEXT("TargetDefensePower Is
Zero!"));
    const float FinalDamageTaken = WeaponBaseDamage /
TargetDefensePower;

    if (FinalDamageTaken > 0.f)
    {
        OutExecutionOutput.AddOutputModifier(
            FGameplayModifierEvaluatedData(
                GetFPSDamageCapture().DamageTakenProperty,
                EGameplayModOp::Override,
                FinalDamageTaken
            )
        );
    }

```

```

    }
}

```

然后在GameplayEffect蓝图中的Execution中将此类加入就构建好了伤害结算的GameplayEffect

由于需要在Player的FireAbility中处理子弹命中，伤害结算的Effect也是在这里Apply到命中的Target上，因此我们需要在Player的Ability中实现函数来构建上述GameplayEffect

```

FGameplayEffectSpecHandle
UFPSPlayerGameplayAbility::MakePlayerDamageEffectSpecHandle(TSubclassOf<UGameplayEffect> EffectClass, float InWeaponDamage)
{
    check(EffectClass);

    FGameplayEffectContextHandle ContextHandle =
    GetFPSAbilitySystemComponentFromActorInfo()->MakeEffectContext();
    ContextHandle.SetAbility(this);
    ContextHandle.AddSourceObject(GetAvatarActorFromActorInfo());
    ContextHandle.AddInstigator(GetAvatarActorFromActorInfo(),
    GetAvatarActorFromActorInfo());

    FGameplayEffectSpecHandle EffectSpecHandle =
    GetFPSAbilitySystemComponentFromActorInfo()->MakeOutgoingSpec(
        EffectClass,
        GetAbilityLevel(),
        ContextHandle
    );

    EffectSpecHandle.Data->SetSetByCallerMagnitude( // 设定
    SetByCallerMagnitude, 向TMap<FGameplayTag, float>中传入武器的伤害

    FPSGameplayTags::Shared_SetByCaller_weaponBaseDamage_Rifle,
        InWeaponDamage
    );
}

```



```
        return EffectSpecHandle;
    }
```

通过上述流程就能够实现将伤害结算施加到目标上

迁移Mixamo动画

mixamo网站上提供了许多免费的动画资产，为了将这些动画资产应用到自己的模型上，需要使用Ureal提供的 `Animation Retarget` 功能，下面介绍具体的流程

添加Root节点(Blender)

Mixamo网站上的模型骨架的根节点为hips(骨盆位置)，但是一般情况下模型都有root节点，位于骨盆的正下方

为了统一，需要给mixamo上的模型加上root节点，为方便通过Blender中的 `Scripting`，添加如下脚本(由Gemini生成)，就可以为模型增加root节点

```
import bpy

def add_root_bone():
    # 1. 获取当前选中的物体
    obj = bpy.context.object

    # 检查是否选中了物体且类型为骨架 (Armature)
    if not obj or obj.type != 'ARMATURE':
        print("错误：请先选中一个骨架 (Armature)! ")
        return

    # 2. 切换到编辑模式 (Edit Mode) 以修改骨骼
    bpy.ops.object.mode_set(mode='EDIT')
    edit_bones = obj.data.edit_bones

    # 3. 检查是否已经存在名为 'root' 的骨骼
    if "root" in edit_bones:
        print("提示：该骨架已经有一个名为 'root' 的骨骼，脚本停止。")
        bpy.ops.object.mode_set(mode='OBJECT')
```

```
return
```

```
# 4. 创建新的 Root 骨骼
```

```
root_bone = edit_bones.new('root')
```

```
root_bone.head = (0, 0, 0)
```

```
root_bone.tail = (0, 0, 10) # 指向 Z 轴上方一点，长度随意，不影响功
```

能

```
# 5. 寻找原来的“根”骨骼 (Mixamo 通常叫 'Hips' 或 'mixamorig:Hips')
```

```
# 逻辑是：找到目前没有父级的骨骼
```

```
original_root = None
```

```
for bone in edit_bones:
```

```
    if bone.parent is None and bone != root_bone:
```

```
        original_root = bone
```

```
        break
```

```
# 如果没找到自动逻辑，尝试直接找 'Hips'
```

```
if not original_root:
```

```
    if "Hips" in edit_bones:
```

```
        original_root = edit_bones["Hips"]
```

```
    elif "mixamorig:Hips" in edit_bones:
```

```
        original_root = edit_bones["mixamorig:Hips"]
```

```
# 6. 建立父子关系
```

```
if original_root:
```

```
    original_root.parent = root_bone
```

```
    print(f"成功: 已将 '{original_root.name}' 的父级设为  
'root'。")
```

```
else:
```

```
    print("警告: 创建了 root 骨骼，但未找到原始 Hips 骨骼，请手动设置父  
级。")
```

```
# 7. 切换回物体模式 (Object Mode)
```

```
bpy.ops.object.mode_set(mode='OBJECT')
```

```
print("完成! ")
```

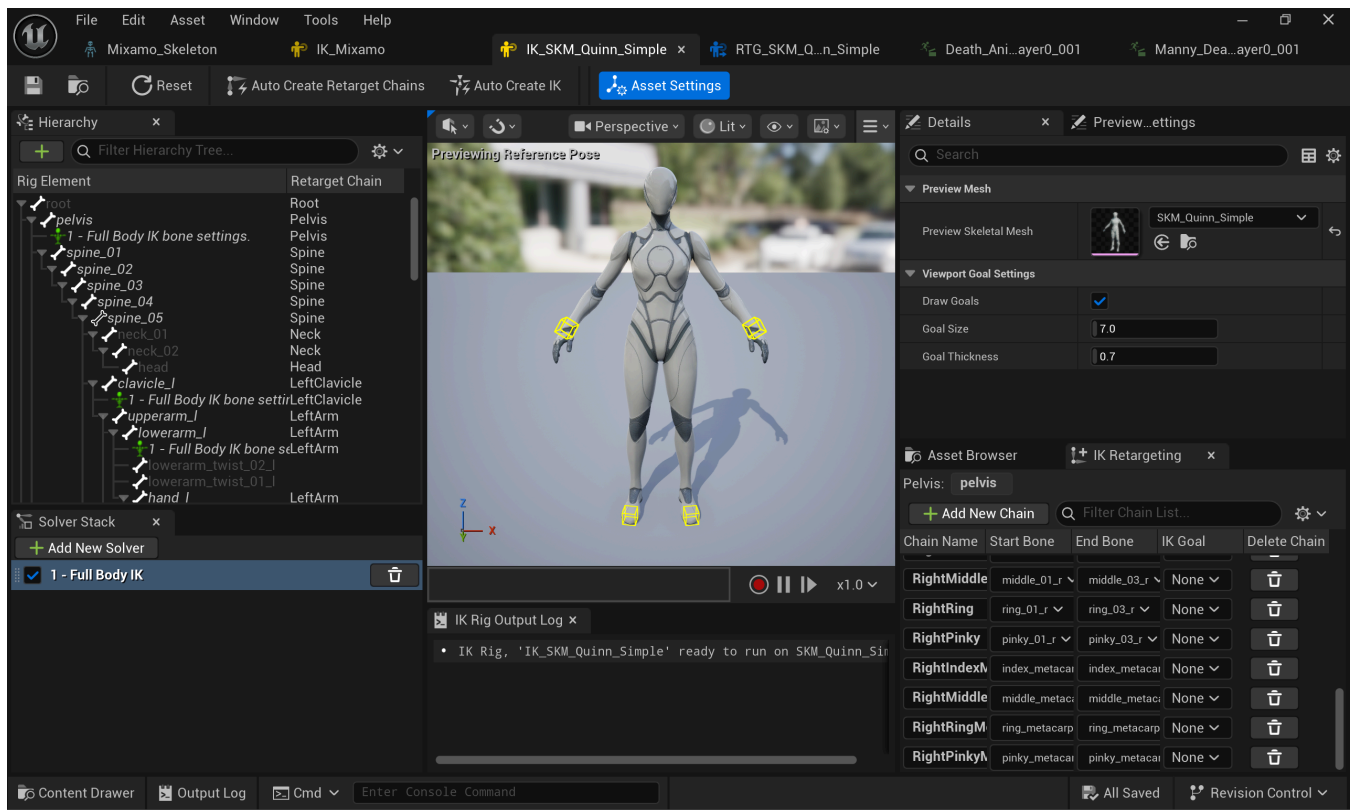
```
# 执行函数
```

```
add_root_bone()
```

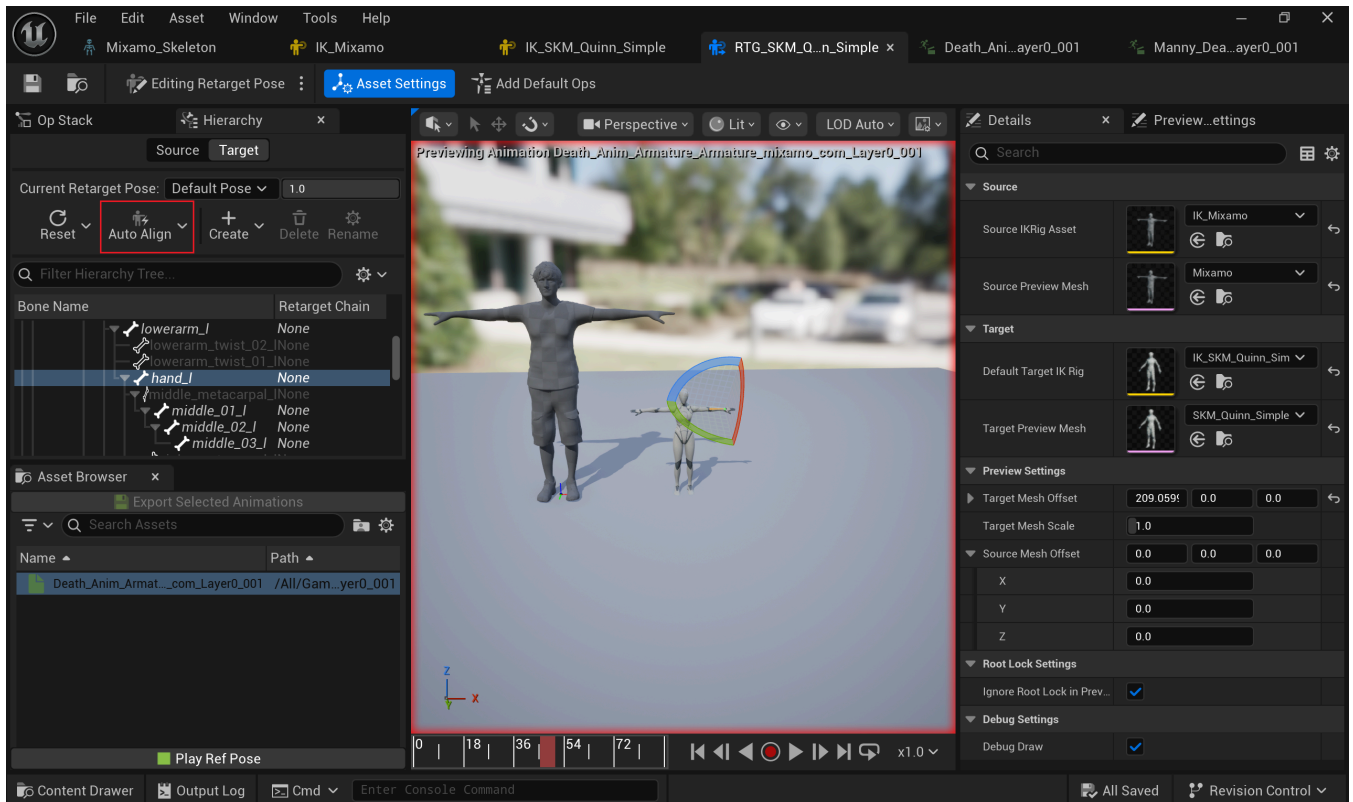
Animation Retarget(Unreal)

首先需要从mixamo网站下载带有base pose的基础模型，由于网站上提供的是T-pose，而Unreal FPS Template提供的是A-pose，因此需要进行调整

为源Skeletal Mesh(即Mixamo中下载的角色模型)和目标Skeletal Mesh(即FPS Template中提供的角色模型)创建IK Rig，然后通过Auto Create Retarget Chain和Auto Create IK为其创建Retarget Chain(相当于——对应的骨骼组)和IK(IK解算的关键部位)



接着创建 IK Retargeter 然后填入Source IK Rig和Target IK Rig，Retargeter里面提供了Auto Align可以直接使用此功能对齐，之后就可以在AssetBrowser中预览Retarget后的动画



通过上述 IK Retargeter 就可以给各种动画资产进行Retarget

参考资料

[UE5使用IK重定向快速实现换皮游戏](#)

[把动画从Mixamo转移到虚幻引擎5【UE5教程】](#)

[The Easiest Way to Retarget Mixamo Animations in Unreal Engine 5](#)

Enemy AI实现

Enemy AI整体流程

对于敌方AI来说主要完成以下的工作

- 首先通过EQS找到合适的射击位置，该位置需要与TargetActor保持一定的距离
- 然后移动到该位置
- 然后激活射击Ability，向目标射击

感知TargetActor

对于敌人AI来说，其需要知道要发起后续行为的目标Actor是谁，在此项目中就是我们所控制的HeroCharacter，那么如何让AIController感知到TargetActor呢？

通过 `UAIPerceptionComponent` 来实现Perception能力，一些配置信息需要通过 `UAISenseConfig_Sight` 来进行配置，如下

```
AI_SenseConfig_Sight = CreateDefaultSubobject<UAISenseConfig_Sight>
("EnemySenseConfig_Sight");
AI_SenseConfig_Sight->DetectionByAffiliation.bDetectEnemies = true;
AI_SenseConfig_Sight->DetectionByAffiliation.bDetectFriendlies =
false;
AI_SenseConfig_Sight->DetectionByAffiliation.bDetectNeutrals =
false;
AI_SenseConfig_Sight->SightRadius = 5000.f;
AI_SenseConfig_Sight->LoseSightRadius = 0.f;
AI_SenseConfig_Sight->PeripheralVisionAngleDegrees = 360.f;
```

由于我们的敌人只需要感知到敌对生物（对于敌人来说敌对生物就是HeroCharacter），因此只需要设置 `bDetectEnemies` 为 `true`，那么如何界定其是敌对还是友方呢？通过实现 `IGenericTeamAgentInterface` 接口，其中定义了 `GetGenericTeamId` 方法用于返回自己的TeamID，和 `GetTeamAttitudeTowards(const AActor& Other)` 用于返回当前对象对其他对象的态度（即是敌对还是友好还是中立）

AIController默认实现了该接口，因此我们需要在构造函数中设置当前Controller的TeamID

```
SetGenericTeamId(FGenericTeamId(1));
```

此外还需要在PlayerController中实现该接口，并返回其TeamId，在实现中，我们将Player的TeamId设置为0

```

AFPSPlayerController::AFPSPlayerController()
{
    HeroTeamID = FGenericTeamId(0);
}

FGenericTeamId AFPSPlayerController::GetGenericTeamId() const
{
    return HeroTeamID;
}

```

由于所有人 (Enemy、Player) 的Controller都实现了该接口 `IGenericTeamAgentInterface`，因此在判断态度时，我们只需要获取其Controller，并将其Cast为 `IGenericTeamAgentInterface`，就可以通过 `GetGenericTeamId` 获取到对方的TeamId，并实现相应的判断逻辑了

```

ETeamAttitude::Type AFPSAIController::GetTeamAttitudeTowards(const
AActor& Other) const
{
    const APawn* PawnToCheck = Cast<const APawn>(&Other);
    const IGenericTeamAgentInterface* OtherTeamAgent = Cast<const
IGenericTeamAgentInterface>(PawnToCheck->GetController()); // 需
要确保其他Pawn的Controller实现了IGenericTeamAgentInterface接口

    if (OtherTeamAgent && OtherTeamAgent->GetGenericTeamId() <
GetGenericTeamId()) // 如果其他Pawn的TeamID小于当前Enemy的TeamID就表
示是敌对的
    {
        return ETeamAttitude::Hostile;
    }

    return ETeamAttitude::Friendly;
}

```

通过上述定义，我们就能够通过AI PerceptionComponent识别到敌对生物，也就是Player，然后绑定AI PerceptionComponent中的回调函数 `OnTargetPerceptionUpdated`

```
EnemyPerceptionComponent-
>OnTargetPerceptionUpdated.AddUniqueDynamic(this,
&ThisClass::OnEnemyPerceptionUpdated); // 绑定回调函数
```

该回调函数的主要作用就是在识别到敌对生物后将识别到的Actor更新到Blackboard中

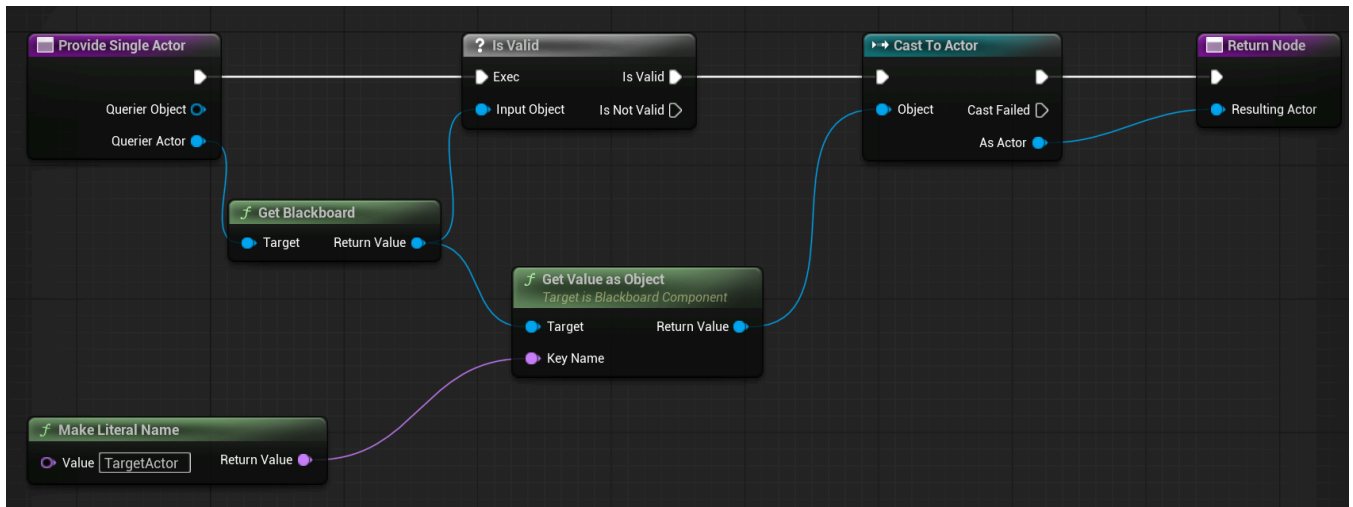
```
void AFPSAIController::OnEnemyPerceptionUpdated(AActor* Actor,
FAIStimulus Stimulus) // 将感知到的Actor更新
{
    if (UBlackboardComponent* BlackboardComponent =
GetBlackboardComponent())
    {
        if (!BlackboardComponent-
>GetValueAsObject(TargetObjectName))
        {
            if (Stimulus.wasSuccessfullySensed() && Actor)
            {
                BlackboardComponent-
>SetValueAsObject(TargetObjectName, Actor);
            }
        }
    }
}
```

EQS查询Shoot Location

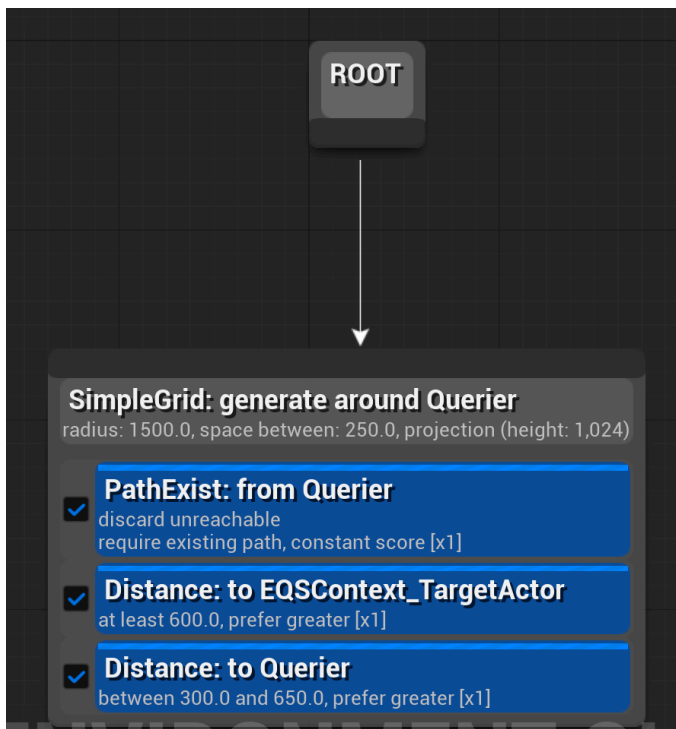
EQS的基本组成为Generator和Context，Generator生成候选点（其也可以作为Context，即默认给定的Context_Item），Context则为查询提供参考信息，默认提供了两个Context，Querier和Item。Querier为查询者，即由AI Controller Possess的Pawn；Item即由Generator生成的候选点

对于当前问题来说，我们由于需要寻找合适的射击位置，我们需要确保该位置和TargetActor有一定的距离，因此需要将TargetActor作为Context信息来进行EQS查询，通过定义继承自 Env Query Context Blueprint Base的蓝图来自定义Context，此蓝图只需要从BlackBoard中获取TargetActor即可，因此其通过

Override Provide Single Actor返回BlackBoard中的TargetActor即可



有了上述Context之后就可以实现用于查询射击位置的EQS，首先需要确保从AI到目标位置的路径存在，其次需要满足该位置与TargetActor有一定的距离，其次为了能够让敌人移动，需要满足该位置与当前AI所在位置有一定的距离，最终实现如下

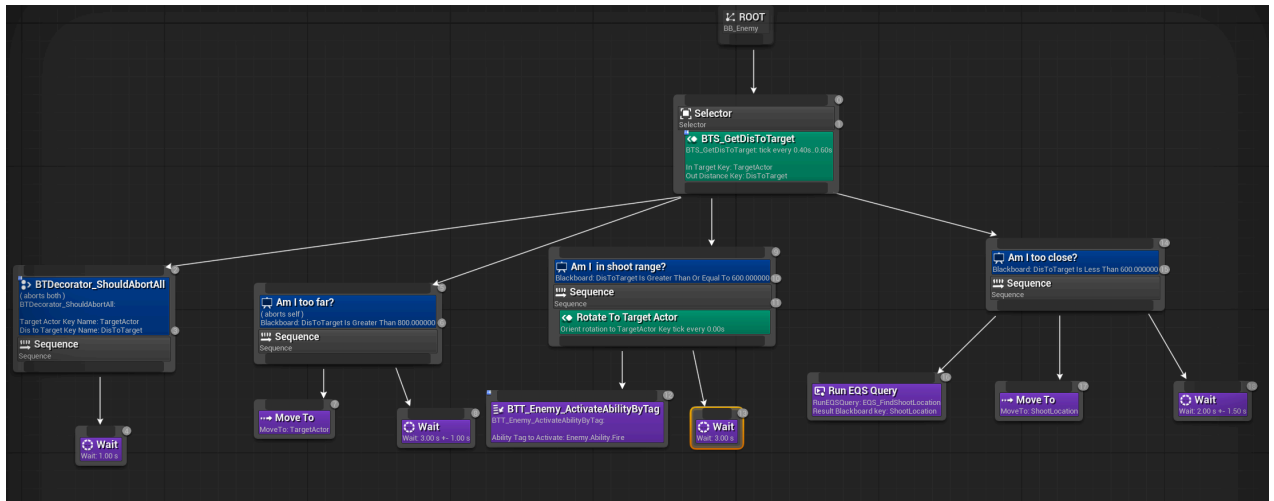


Behavior Tree整体结构

Behavior Tree由Selector分出的四个分支组成，Selector挂了一个Service用于实时更新DistanceToTarget

1. 优先级最高的最左边的分支用于处理特殊的情况
 1. 初始时DistanceToTarget变量为0会导致冗余执行EQS分支，所以需要在此分支等待到DistanceToTarget更新为有效值

2. 当前AI或者目标死亡
2. 当距离目标太远，就会向目标走去
3. 当处于射击范围内，就会通过自定义Task激活射击能力向目标开火
4. 当距离目标太近时，会执行EQS找到合适的射击位置，然后向该位置移动



网络同步

表现与逻辑分离

类似于Web应用开发中前后端分离的思想，对于一些数值、属性计算应该由服务端完成并同步给客户端，客户端在同步到最新的属性信息后通过OnRep回调函数处理相应的表现层逻辑，比如更新UI，生成特效，播放音效，改变动画等

GameplayEvent

GameplayEvent不会像GameplayTag那样通过ASC进行同步，也就是说需要保证SendGameplayEvent和WaitGameplayEvent在同一端进行，要么都在客户端，要么都在服务端

Unreal RPC

如果出现了在客户端想要向服务端发送GameplayEvent的情况，该如何处理，解决方法就是通过RPC(Remote Procedure Call)，就是能够让我们像函数调用一样调用远端的某个函数

通过在UFUNCTION中加上 `server` 就可以定义一个客户端调用的RPC，可以通过加上 `Reliable` 来指示该RPC是可靠传输（类似TCP与UDP），然后在 `.cpp` 文件中给出 `FUNCTIONNAME_IMPLEMENTATION`，即加上 `_IMPLEMENTATION` 后缀，给出Server端的实现

本项目通过RPC实现了玩家的 `Fire`，因为服务端需要从客户端获取客户端的 `Controller` 的参数来生成子弹，因此需要通过RPC来传递客户端的参数

GameMode & GameState

GameMode仅在服务端，因此不应该在GameMode定义一些Replicate变量，向客户端同步，GameMode一般用来实现一些Tick逻辑来更新GameState

GameState在客户端和服务端都有，因此一般在此处定义一些Replicate变量，比如用于表示游戏状态的，通过同步给客户端告知客户端不同的游戏阶段，使得可以客户端可以根据此信息更新UI

游戏整体流程简述

AI Enemy

- 通过DataTable配置生成的敌人情况
- 敌人生成后通过BTService，随机选取感知到的Target进行攻击

Player

- 玩家有基本的控制，可以移动、跳跃
- 按下 `E` 可以装备武器 `Rifle`，然后点击左键可以开火，按下 `R` 可以装填弹药
- 装备武器状态时，按下 `E` 可以卸下武器
- 玩家击杀敌人后会在屏幕右上角显示杀敌数量

游戏结束状态

- 当玩家杀完所有波次的敌人后会进入游戏结束状态
- 或者，当所有玩家死亡时，同样会进入游戏结束状态

- 此时，客户端会展示游戏结算界面，显示当前对局中所有玩家的杀敌数量

调试技巧

Ability System调试

按下引号" 按键（回车旁边的按键），可以进入Gameplay Debugger调试界面，通过按下小键盘的数字键可以显示自己需要的调试信息

此外，通过在命令行中（通过`键打开命令行）输入 `ShowDebug AbilitySystem` 展示Ability System的调试信息

Visual Studio Attach To Process

在开发的过程中，遇到了卡死的问题，一般就是程序陷入了死循环导致的，通过Visual Studio提供的Attach To Process功能就能够通过Call Stack信息找到程序卡在哪一行指令上

1. 启动Unreal Editor（可以不需要通过Visual Studio启动，通过.uproject独立启动也行）
2. 在Visual Studio中 `Debug>Attach To Process`，将 `UnrealEditor.exe` Attach 过去
3. 然后复现卡住的情况
4. 回到Visual Studio点击Break All（两根竖线 `||` 的按钮）
5. 然后在Call Stack窗口中查看最后调用的指令

Bug及解决方案

UComponent为空导致的空指针访问(Access Violation)

BUG: 加载Enemy的GameplayEffect（主要原因在于加载与CurrentHealth相关的GameplayEffect）时导致游戏的崩溃。

在实现Enemy时，由于AttributeSet(声明在BaseCharacter类中)中定义了如下的代码，因此其需要通过GetPawnUIComponent方法来获取PawnUIComponent，BaseCharacter虽然实现了IPawnUIInterface接口，但是其没有PawnUIComponent成员，其函数也只是默认实现，即返回nullptr，因此需要派生类自己声明自己的PawnUIComponent，并override相应的函数，使得其能够获得有效的PawnUIComponent，如果没有定义就会出现空指针访问的错误(Access Violation)，因此在处理指针是需要额外注意，必要时加上check或者checkf，使得程序崩溃时可以知道是哪里的的问题

```
void UFPSAttributeSet::PostGameplayEffectExecute(const
FGameplayEffectModCallbackData& Data)
{
    if (!CachedPawnUIInterface.IsValid())
    {
        CachedPawnUIInterface =
TweakInterfacePtr<IPawnUIInterface>(Data.Target.GetAvatarActor());
    }
    checkf(CachedPawnUIInterface.IsValid(), TEXT("%s didn't
implement IPawnUIInterface"), *Data.Target.GetAvatarActor()-
>GetActorNameOrLabel());
    •
    UPawnUIComponent* PawnUIComponent = CachedPawnUIInterface-
>GetPawnUIComponent();
    •
    if (Data.EvaluatedData.Attribute ==
GetCurrentHealthAttribute()) // 生命值发生变化之后，需要通过
PawnUIComponent对Delegate进行Broadcast
    {
        const float NewHealth = FMath::Clamp(GetCurrentHealth(),
0.f, GetMaxHealth());
        SetCurrentHealth(NewHealth);
        PawnUIComponent-
>OnCurrentHealthChanged.Broadcast(GetCurrentHealth() /
GetMaxHealth());
    }
}
```

此外在BaseClass的PossessedBy函数中，初始化了AbilitySystemComponent的信息(AvatarActor)，在派生类中需要override该函数，用于加载StartupData，但是需要通过静态的方式来调用基类的PossessedBy函数，通过

`Super::PossessedBy(NewController)` 来调用，但是在实现时没有调用，导致AbilitySystemComponent的信息无效

Enemy的GameplayEffect无法生效问题

BUG: Enemy虽然在StartupData中传入了自己的GameplayEffect，但是却无法生效，其Apply的依然是Player定义的GameplayEffect，导致两者的属性完全一致，即Enemy和Player共用一个GameplayEffect

貌似是showdebug abilitysystem仅会显示客户端的属性，最好使用  按键进行AbilitySystem的Debug

UnrealEditor中设置的初值没法修改成员变量

在Weapon类中，我设置了MaxAmmo，并将其UPROPERTY设置为EditDefaultsOnly，即可以在Unreal Editor中修改其默认值，此外我还定义了CurrentAmmo私有成员，用于记录当前的子弹数量

我在类中给了MaxAmmo默认初值，并在构造函数中将CurrentAmmo设置为MaxAmmo，但是当我在UnrealEditor中修改了MaxAmmo的初值后，CurrentAmmo的值却依然没有改变，依然是类中默认的初值

这跟Unreal C++的生命周期有关

1. 首先会进行构造函数
2. 然后序列化/属性加载，将UnrealEditor的属性加载到成员上
3. BeginPlay/OnConstruction：游戏开始

所以，在构造函数中赋值时机过早，应该在BeginPlay中进行赋值

Ability之间的冲突问题

虽然在Ability的Block中配置了需要Block的Ability，但是在此Ability激活时去试图激活Blocked Ability，虽然Blocked Ability不会激活，但是却会使得当前的Ability提前中止(提前进入End Ability)

此外，如果不设置Block Ability，重复上述步骤，则会直接激活另一个Ability

原因是由于在给Ability绑定输入事件时，对于Release事件的处理，我在Release时直接通过如下的方式将Ability Cancel掉了，导致Ability会提前结束

```
void UFPSAbilitySystemComponent::OnAbilityInputReleased(const
FGameplayTag& InInputTag)
{
    if (!InInputTag.IsValid())
    {
        return;
    }
    for (const FGameplayAbilitySpec& AbilitySpec :
GetActivatableAbilities())
    {
        if
(AbilitySpec.GetDynamicSpecSourceTags().HasTagExact(InInputTag)
&& AbilitySpec.IsActive())
        {
            CancelAbilityHandle(AbilitySpec.Handle);
        }
    }
}
```

供选择的BlackBoard Key列表中仅显示SelfActor

在BehaviorTree中使用MoveTo Task让AI向目标Actor移动，在选择Blackboard Key时，仅有一个默认选项即SelfActor，虽然在Blackboard中添加了自定义的Object类型的Key `Target Actor`，但是却没有出现在选项中

原因是由于MoveTo仅接受以下两种作为目标

- Vector类型，即给出具体的位置
- Object类型且Base Class设置为Actor，给出目标Actor

由于没有在Blackboard中将TargetActor的Base Class设置为Actor导致其没有出现在可选择的列表中

AI角色一直处于Idle状态

当AI执行Move To移动到目标位置时，角色的动画始终处于Idle状态，初步判断应该这是由于动画状态机的转换条件中的某个变量出了问题

由于AnimInstance通过如下的条件来判断是否需要从Idle转化为移动状态

```
bShouldMove = OwingMovementComponent->GetCurrentAcceleration().SizeSquared() > 0.f  
            && GroundSpeed > 0.01f;
```

运行时查看，发现AI角色的加速度就算是移动时依旧为0

这是由于玩家输入驱动的移动和AI寻路驱动的移动有区别

- 玩家输入驱动：相当于在输入方向上施加一个力，由牛顿第二定律 $F=ma$ 知，此时玩家角色会产生加速度
- AI寻路驱动：则是由AI Controller直接设置其速度(RequestedVelocity)，一般情况下加速度为0

要使其有加速度，在角色蓝图中开启Use Acceleration for Paths就可以使得其能够有加速度